

Chapter 11

Lorraine Gaudio

Version March 11, 2026



BOISE STATE UNIVERSITY

Contents

1	Overview	3
2	Quarto	4
2.1	R Notebook & Quarto	4
2.2	The YAML Header	5
3	& Packages	6
4	The Pipe (%>%)	7
5	Selecting Columns	8
5.1	Positive Selection	8
5.2	Negative Selection	8
5.3	Piping with Select	8
5.4	Subset with Select	8
6	Filtering Rows	10
6.1	Single Condition	10
6.2	Piping with Filter	10
6.3	Multiple Conditions with AND	11
6.4	Multiple Conditions with OR	11
7	filter() AND select()	13
8	One Application	14
9	Duplicates & Sampling	15
10	Extra Practice	16
11	Summary	17
12	Chapter Terms	18
13	& Practice Space	19
13.1	Task 0	19
13.2	Task 1	19
13.3	Task 2	19
13.4	Task 3	19
13.5	Task 4	20
13.6	Task 6	20

13.7	Task 7	20
14	Save and Upload	21
15	Today you practiced:	22
16	Notes	23
16.1	Learning Skill	23

1. Overview

In Chapter 10, you learned about subsetting, extraction, and insertion in data frames using base R functions. This Chapter, you will learn easier subsetting with dplyr (`select()` & `filter()`). The dplyr package simplifies column and row extraction, making code shorter and easier to read than base-R brackets. Mastering `select()`, `filter()`, and the pipe `%>%` will speed up every day tasks.

Lesson Overview

By the end of Chapter 11 you will be able to:

1. ☐ Remember – List what `select()`, `filter()`, and `%>%` do.
2. ☐ Understand – Explain how `select()` (columns) differs from `filter()` (rows).
3. ☐ Apply – Use `select()` and `filter()` to subset a data frame.
4. ☐ Analyze – Chain multiple verbs with `%>%` to answer a real question.
5. ☐ Evaluate – Decide when dplyr is preferable to base-R brackets.

Keep these goals in mind as you move through each section.

2. Quarto

In earlier chapters, you used an R Notebook as a reproducible thinking document: code, output, and memo-style writing in one place. Quarto keeps that same basic workflow, but the file is usually a .qmd file instead of an .Rmd file, and chunk controls are written in YAML-style lines that begin with `#|` inside the code chunk. Quarto is the next-generation version of R Markdown, and RStudio supports creating Quarto documents from File > New File > Quarto Document... and rendering them with the Render button.

What are the benefits of Quarto over the R Notebook?

- Quarto supports multiple languages (R, Python, Julia, Observable JavaScript) in the same document. R Notebooks only support R.

2.1 R Notebook ↔ Quarto

Quarto will be **similar** to an R Notebook in several ways. You still write normal paragraphs outside code chunks. You still run R code inside chunks. You still use Markdown headings to create an outline. You still render the document to create a readable report.

What changes is the file extension is .qmd, the YAML uses format: instead of the old output: style from R Markdown and the chunk controls are written with `#|` lines at the top of the chunk.

□ Create a Quarto File

1. For those of you who **have GitHub** and a course project, navigate to your course folder and open your course project. This will open RStudio. Click “Pull” in the Git pane to get the latest version of the course materials. If you **don’t have GitHub** or a course project, just open RStudio and set your working directory to your course folder.
2. Open a new Quarto document in your course folder and save it as: chapter11_notes.qmd (File > New File > Quarto Document).

Just like an R Notebook template, a new Quarto file will open with sample text. Start fresh:

- Keep the YAML header at the top.
- Delete the example body text below it.
- Replace that content with your own notes, code, and memos.

2.2 The YAML Header

At the top of the document is the YAML header. This is document metadata, not R code. In Quarto, the header is still surrounded by three dashes (---) and uses key: value pairs. Official Quarto examples show fields such as title, author, date, and format. Quarto also supports the dynamic date keyword today.

```
---
title: "Chapter 11 Notes"
subtitle: "dplyr Subsetting"
author: "YOUR NAME"
date: today
format:
  html:
    code-fold: true
execute:
  warning: true
  message: true
---
```

What each part does:

- title: gives your document a title.
- author: puts your name on the document.
- date: today inserts the current local date automatically. [Quarto Dates and Date Formatting](#) has more options for date formatting and dynamic date keywords.
- format: html tells Quarto to render HTML output.
- code-fold: true lets readers collapse code in the HTML. You can change this to false if you want code to always show. Notice that the true and false values are not capitalized. This is a YAML convention and different from R's TRUE and FALSE.
- execute: sets default chunk behavior for the whole document.

3. ☒ Packages

□ Recall that we need to download packages **once**. If needed, install the required packages we will use in this lesson. After installation, comment `#` out the `install.packages` code.

```
#install.packages("dslabs")  
install.packages("dplyr")
```

After installing the package, you need to load it into your R session using the `library()` function. Each time you open R, you will need to “library” the packages. This makes the functions in the package available for use. Type the following code in a new code chunk and run.

□ Pro tip: You can load multiple packages in one `library()` call by separating package names with commas.

```
library(dslabs, dplyr)
```

4. The Pipe (%>%)

□ The pipe operator passes the result of the left side into the first argument of the right side, reading like “and then”. It cuts repetition and keeps code in logical order.

```
# □ Test  
mtcars %>%      # start with data  
head()         # □ shows first six rows
```

□ Link Concepts: What other ways have you learned to write code that will view the first six rows of a data frame? Write this code and a memo to demonstrate learning skills.

5. Selecting Columns

- The GOAL of `select()` is to keep or drop specific **columns**.
- The SYNTAX is `select(data, col1, col2, ...)`

5.1 Positive Selection

- Goal: keep specific columns by name.

```
# □ Test  
select(mtcars, mpg, cyl, hp) # keep three columns
```

5.2 Negative Selection

- Goal: remove columns with a leading -.

```
select(mtcars, -gear) # drop one column
```

5.3 Piping with Select

- Goal: Let's add a pipe!

```
mtcars %>%  
  select(mpg, cyl, hp) # easier to read
```

5.4 Subset with Select

We'll mix it up by using a different data set, `gapminder` from `dslabs`. This data set contains information about countries over time.

```
?gapminder
```

- Goal: Create a copy of the `gapminder` dataset that contains only specific columns!

```
# □ Test:  
I_Demo <- gapminder %>%  
  select(country, year, infant_mortality, continent)
```

```
# □ Verify  
head(I_Demo)
```

□ Explain: A new dataset called I_Demo was created from the gapminder dataset that only contains the columns country, year, infant_mortality, and continent.

□ Practice: fill in the blank prompt

You try to design your own subsetting operation.

□ Goal: Write an example that selects only the columns you choose.

```
# □ Test:  
Your_Choice <- gapminder %>%  
  select(____, ____, ____ ) # choose columns  
  
# □ Verify
```

□ Explain: What columns did you choose and why? Create a memo note, demonstrate learning skill(s) used.

6. Filtering Rows

- The GOAL of `filter()` is to keep or drop specific **rows** based on conditions.
- The SYNTAX is `filter(data, condition1, condition2, ...)`

6.1 Single Condition

- `filter()` uses logical operators!

```
# □ Test:  
filter(mtcars, mpg >= 25)      # high mileage cars
```

6.2 Piping with Filter

We'll mix it up by using a different data set, murders from dslabs. This data set contains information about gun murders in the US.

```
?murders
```

- Goal: Create a copy of the murders dataset that excludes Washington state.
- Prediction: 50 states

```
# □ Test:  
nrow(murders)
```

- Explain: ... Wait, what? ... □ Question: What are these 51 states?

```
# □ Test:  
unique(murders$state) # 51 states
```

- Explain: Ohhhhhh! "District of Columbia" is included in the dataset.

Back to the □ Goal: Create a copy of the murders dataset that excludes Washington state.

```
# □ Test:  
No_Washington <- murders %>%  
  filter(state != "Washington")
```

```
# □ Verify
nrow(No_Washington) # 50 states
```

- Explain: A new dataset called No_Washington was created from the murders dataset that excludes Washington state.
- Review: What is the difference between == and !=?
- Break Things! What happens when you use = instead of ==? or != instead of !=? Create a memo note, demonstrate learning skill(s) used.

6.3 Multiple Conditions with AND

- In filter(), commas = AND (&).
- Goal: Let's filter with two conditions and while we are at it, let's add a pipe!
- Question: How many cars have *both* mpg ≥ 25 *and* 4 cylinders?

```
# □ Test:
mtcars %>%
  filter(mpg >= 25, cyl == 4) # two conditions
```

- Verify: How many cars have *both* mpg ≥ 25 *and* 4 cylinders? What code would you use to verify the success of this filter? Create a memo note, demonstrate learning skill(s) used by writing and running the code.

6.4 Multiple Conditions with OR

- In dplyr::filter(), the OR operator is the same as base R | or %in%.

6.4.1 |

- Goal: Let's filter with two conditions using OR (|).
- Question: How many cars have *either* mpg ≥ 25 *or* 4 cylinders?

```
# □ Test:
mtcars %>%
  filter(mpg >= 25 | cyl == 4) # two conditions
```

- Verify: How many cars have *either* mpg ≥ 25 *or* 4 cylinders? What code would you use to verify the success of this filter? Create a memo note, demonstrate learning skill(s) used by writing and running the code.

6.4.2 %in%

□ Goal: The %in% operator is used to check if a value is in a set. It's like “is this value one of these?”

```
# □ Test  
filter(mtcars, cyl %in% c(4, 6))    # 4 or 6 cylinders
```

7. filter() AND select()

- The GOAL of combining filter() and select() is to **subset** a data frame by both rows and columns.
- Best practice: filter rows first, then select columns.
- The SYNTAX is data %>% filter(condition) %>% select(col1, col2, ...)
- We'll mix it up by using a different data set, us_contagious_diseases from dslabs. This data set contains information about contagious diseases in the US.

```
?us_contagious_diseases
```

```
# Test  
us_contagious_diseases %>%  
  filter(state == "Alaska", year >= 1980) %>% # rows  
  select(year, disease, count)                # columns
```

- Explore and Play: What are the unique diseases in this Alaska subset? Use skills you developed in previous lessons to explore this data set. Compare filter and select with subsetting with brackets []. Create a memo note, demonstrate learning skill(s) used.

8. One Application

□ Let's say you have the research question, □ “Which automatic cars (`am == 0`) in `mtcars` have horsepower > 150 ? □ Goal: Your report should show only `hp` and `mpg`.

```
#□ Test  
mtcars %>%  
  filter(am == 0, hp > 150) %>%  
  select(hp, mpg) %>%  
  head()
```

□ Explain:

1. `filter()` keeps matching rows.
2. `select()` trims to needed columns.
3. `head()` previews results. Bonus: Add row count for exploration.

9. Duplicates & Sampling

Sometimes, you may need to remove duplicate rows or sample a subset of rows. `unique()` removes duplicate rows; `slice_sample()` picks random rows. □ Type `?slice_sample()` to learn more about this dplyr function.

□ Goal: We'll mix it up by using a different data set, `movielens` from `dslabs`. This data set contains information about movies and ratings.

```
?movielens
```

```
nrow(movielens) # how many rows at start?
```

```
# □ Test
```

```
Movie_Night <- movielens %>%  
  filter(rating >= 4) %>% # keep high ratings  
  select(title, rating) %>% # keep title and rating  
  unique() # remove duplicates
```

```
# □ Verify
```

```
nrow(Movie_Night) # how many rows after filter, select, and unique?
```

□ Explain: Filter kept only the rows with ratings 4 or higher. Select kept only the title and rating columns. Unique removed duplicate rows, so each movie title appears only once.

□ Question: What movie should I watch on movie night?

□ Goal: Let's randomly sample 10 movies with ratings four or higher.

```
set.seed(420)
```

```
# □ Test
```

```
Movie_Night %>%  
  slice_sample(n=10) # random sample of 10 rows
```

□ Comment: What movie would you choose?

□ Explore and Play: What would happen if we didn't use `unique()`? Why might `unique` be useful for our question? Create a memo note, demonstrate learning skill(s) used.

10. Extra Practice

☐ Practice: Demonstrate learning skill(s) by completing this bonus practice. Fill the blanks. Use the gapminder dataset.

```
____ %>%  
  filter(life_expectancy >= quantile(life_expectancy, ____)) %>%  
  select(country, ____, life_expectancy) -> Top_LifeExp  
View(Top_LifeExp)
```

- ☐ Explain: What is the -> Top_LifeExp do in the code above?
- ☐ Question: What is the highest life expectancy in this subset?
- ☐ Question: How many countries are in the Top_LifeExp data frame?
- ☐ Question: What is the average life expectancy in this subset?

11. Summary

12. Chapter Terms

13. ☒ Practice Space

Replace each _____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment. When finished, save **BOTH** your script (which should include your lesson notes and completed assignment) and your .RData workspace and upload to Canvas Assignment 8 in Module 8.

13.1 Task 0

- ☐ Make sure dplyr and dslabs packages are attached so their functions / datasets load.

13.2 Task 1

Column Selection

- ☐ Create **C_Overload** that keeps only year and carbon_emissions from the temp_carbon data frame. Use select() with or without %>%.

```
_____ <- _____
```

13.3 Task 2

Single-Column Extraction

- ☐ Pipe the stars dataset in dslabs through select() to keep only the star column. Name the result **star_names**.

```
_____ <- _____
```

13.4 Task 3

Row Filtering

- ☐ From murders, remove every row where state equals "Florida". Name the object **No_Florida** (any method is fine).

```
_____ <- _____
```

13.5 Task 4

Multiple Conditions

☐ From mtcars, use `filter()` + `%>%`, pull cars in mtcars that are manual, have 8 cylinders, and have `qsec < 15`. Store as **Fast_n_Furious**.

```
summary(mtcars$qsec) # check qsec range
```

Is `qsec < 15` within range?

```
_____ <- _____
```

13.6 Task 6

OR Logic + Column Slice

In ☐ mtcars, show only rows where `disp > 250` OR `disp < 160` and display just the first three columns.

```
_____ <- _____
```

13.7 Task 7

Reflection

☐ Write a few sentences comparing dplyr verbs over base-R brackets (from lesson 6 & 7) for subsetting.

14. Save and Upload

0. Self-check checklist:

For the assignment portion, your environment should contain the following items:

- C_Overload exists and has only year and carbon_emissions.
 - star_names exists and has one column named star.
 - No_Florida has 0 rows with state == 'Florida'.
 - Fast_n_Furious with rows that only contain manual, 8 cylinders, and where qsec < 15.
 - **All code in your Quarto document runs top-to-bottom without errors.**
1. You will be submitting **both** the Quarto document and the workspace file. The workspace file saves all the objects in your environment that you created in this lesson. You can save the workspace by running the following command in a code chunk of the Quarto Document document:

```
save.image("Assignment8_Workspace.RData")
```

Or you can click the “Save Workspace” button in the Environment pane.

☐ **Always save the R documents before closing.**

2. Find the assignment in this week’s module in Canvas and upload **both** the .qmd and the workspace file (RData).

15. Today you practiced:

- Installed / loaded packages for extra tools.
- Used the pipe `%>%` to chain steps left-to-right.
- Selected columns with `select()` (including negative selection).
- Filtered rows with `filter()` using logical tests and `%in%`.
- Combined verbs to answer data questions concisely.

□ Great job! Keep practicing piping and you'll soon write clear, readable R code!

16. Notes

16.1 Learning Skill

LS: Core Review course content Advanced Curiosity Skill Highlight

Citing resources

- (Curiosity highlight): “data scavenger hunt” + citation + one curiosity question answered with a quick code experiment (not a paragraph of vibes).
- Courage: Students can share imperfect work in a discussion/lab setting and revise based on feedback.

Maybe Module 6: Autonomy: - “trying alternatives” (base R vs dplyr; indexing vs verbs) and documenting what the output shows. - Students can manage their workflow independently: organized folders, naming, saving, and submitting correct artifacts without resource rescue. - Students can set boundaries on AI use consistent with instructions and explain their choice in a memo (what they delegated vs what they did themselves). - Students can select a method and justify it (e.g., base R indexing vs dplyr select/filter) using output evidence. “I chose this approach because...” memo

Maybe Module 7: Open-mindedness - Students can implement two approaches to the same task (e.g., base R vs dplyr) and compare outputs and readability. - Students can revise their approach when evidence shows a better method (not because it’s trendy). - Students can evaluate tool fit (including AI systems) for a specific goal and explain tradeoffs.